



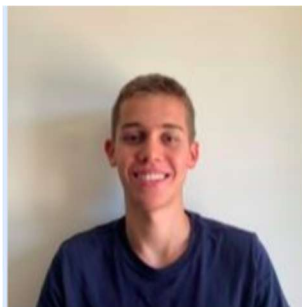
**Universidade do Minho**  
Escola de Engenharia

# **Sistemas Distribuídos**

## **Trabalho Prático**

**Grupo 59**

9 de janeiro de 2026



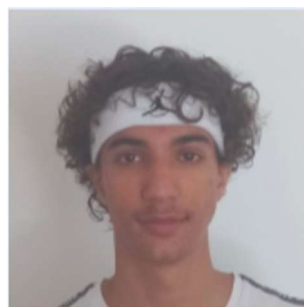
**Diogo Costa**  
A107328



**Gonçalo Costa**  
A107381



**Guilherme Marques**  
A106805



**Lourenço Martins**  
A106849

# Índice

|                                                                |   |
|----------------------------------------------------------------|---|
| 1. Introdução .....                                            | 3 |
| 2. Arquitetura do Sistema .....                                | 3 |
| 3. Funcionalidades .....                                       | 3 |
| 3.1. Autenticação e Registo .....                              | 3 |
| 3.2. Registo de Eventos.....                                   | 3 |
| 3.3 Agregação de informação .....                              | 3 |
| 3.4 Filtragem de Eventos .....                                 | 3 |
| 3.5 Notificações (Blocking Operations).....                    | 4 |
| 4. Servidor.....                                               | 4 |
| 4.1 Gestão de memória .....                                    | 4 |
| 4.2 Persistência e Streaming .....                             | 4 |
| 4.3 Sincronização Granular.....                                | 4 |
| 5. Biblioteca do Cliente.....                                  | 4 |
| 6. Interface do Utilizador .....                               | 5 |
| 7. Avaliação de desempenho.....                                | 5 |
| 7.1 Cenários de Teste .....                                    | 5 |
| 7.1.1 Ingestão Massiva Aleatória.....                          | 5 |
| 7.1.2 Ingestão em Hotspot.....                                 | 5 |
| 7.1.3 Leitura Intensiva (Agregação).....                       | 5 |
| 7.2 Análise dos Resultados .....                               | 5 |
| 7.2.1 Impacto da Concorrência na Escrita (Cenários 1 e 2)..... | 6 |
| 7.2.2 Comportamento na Leitura (Cenário 3) .....               | 8 |
| 8. Conclusão.....                                              | 9 |

# 1. Introdução

O objetivo deste projeto foi implementar e desenhar um sistema distribuído cliente-servidor que faz o registo de eventos em séries temporais e agrega informação, relativa à venda de produtos. A informação é mantida no servidor e acedida remotamente. O programa também suporta o acesso concorrente de múltiplos clientes e garante consistência e consistência dos dados.

Este relatório descreve a arquitetura do sistema, o protocolo de comunicação desenhado e justifica as principais decisões tomadas para cumprir todos os requisitos, como a gestão eficiente de memória e concorrência.

Este relatório contém mais páginas que o inicialmente solicitado, devido à inclusão de algumas imagens detalhadas das avaliações de desempenho.

## 2. Arquitetura do Sistema

O sistema segue uma arquitetura clássica Client-Server e comunica através de sockets TCP.

O servidor segue um modelo de uma thread por cliente ao nível da conexão, onde cada socket TCP é gerido por uma thread dedicada (`ServerWorker`) responsável por demultiplexar o tráfego de entrada. De maneira a suportar clientes multi-threaded, o `ServerWorker`, ao receber um pedido, lança uma thread de execução para processar essa tarefa específica, permitindo a liberdade de receber novos pedidos.

## 3. Funcionalidades

### 3.1. Autenticação e Registo

Permite o registo de novos utilizadores e a autenticação segura antes de realizar qualquer operação. O servidor mantém um registo de credenciais protegido por exclusão mútua.

### 3.2. Registo de Eventos

Suporta a inserção de eventos de venda (Produto, Quantidade, Preço) e a operação de avanço temporal (na interface: `New day`).

### 3.3. Agregação de informação

Permite consultas sobre o histórico, calculando métricas como a quantidade total vendida de um produto, o volume de vendas de um produto (total faturado), o preço médio de venda de um produto e o preço máximo de venda de um produto.

### 3.4. Filtragem de Eventos

Permite listar todos os eventos de um conjunto de produtos num dia específico do passado, utilizando serialização compacta para minimizar o tráfego de rede.

## 3.5 Notificações (Blocking Operations)

Suporta operações bloqueantes onde o cliente fica à espera de condições específicas. Existem dois casos implementados:

- **Vendas Simultâneas:** O cliente é notificado quando dois produtos específicos são vendidos no mesmo dia (notificação também acontece se as vendas já aconteceram antes do pedido de notificação).
- **Vendas Consecutivas:** O cliente é notificado após N vendas seguidas de um produto nesse dia, após o pedido de notificação.

## 4. Servidor

A classe central `TimeSeriesDB` gere o armazenamento e a sincronização. Foram portanto tomadas as decisões de desenho seguintes:

### 4.1 Gestão de memória

Foi implementada uma gestão hierárquica de dados. O servidor aceita um parâmetro `S` (limite de dias em RAM). Mantém-se um mapa `daysInMemory` para acesso rápido. Quando o número de dias em memória excede `S`, o sistema expulsa o dia mais antigo em memória e remove-o da RAM.

### 4.2 Persistência e Streaming

Para garantir a durabilidade, o dia corrente é gravado em disco imediatamente após avançarmos para o próximo dia. Para haver o processamento de agregações sobre dias antigos que residem no disco (que podem ser gigantes) o método `aggregateFromDisk` lê o ficheiro binário sequencialmente, processa cada evento e descarta-o de imediato, garantindo um uso de memória constante e baixo.

### 4.3 Sincronização Granular

Em vez de um `Lock` global, foi utilizado um mapa de locks (`dayLocks`), associando um `ReentrantReadWriteLock` a cada dia. Isto permite paralelismo máximo na leitura (vários clientes consultam o mesmo dia simultaneamente) e escrita paralela em dias diferentes. O bloqueio de escrita é adquirido apenas para inserções do dia corrente.

## 5. Biblioteca do Cliente

A biblioteca (`ClientLibrary`) abstrai a complexidade da rede e implementa a lógica de multiplexagem necessária para suportar aplicações multi-threaded.

Para permitir que várias threads partilhem a conexão TCP sem bloqueio, foi criada uma classe `TaggedConnection`, classe responsável por enviar e receber frames atômicos com tags. Para além disso, no `Demultiplexer`, uma thread dedicada lê continuamente do socket e coloca as respostas em filas de espera (`BlockingQueue`) associadas a cada tag. Quando uma thread do cliente faz um pedido, gera uma tag, envia a mensagem e bloqueia numa `Condition`. É acordada apenas quando o `Demultiplexer` recebe a resposta correspondente à sua tag.

## 6. Interface do Utilizador

A interface desenvolvida (`ClientUI`) demonstra todas as capacidades do sistema. Implementa um menu interativo que permite lançar operações síncronas (como consultas) e assíncronas (como subscrições de notificações) em simultâneo, evidenciando a capacidade da biblioteca de gerir múltiplos fluxos de comunicação concorrentes.

## 7. Avaliação de desempenho

Para validar a solução, desenvolvemos o módulo de teste `TimeSeriesRunner`, submetendo o sistema a três cenários de carga distintos, variando o número de clientes concorrentes. Estes testes serviram para avaliar o comportamento dos mecanismos de sincronização.

### 7.1 Cenários de Teste

#### 7.1.1 Ingestão Massiva Aleatória

Simulamos múltiplos clientes a inserir eventos aleatórios no dia corrente com nomes de produtos distintos. Isto testa o impacto do `WriteLock` no acesso concorrente ao dia.

#### 7.1.2 Ingestão em Hotspot

Múltiplos clientes inserem eventos rapidamente para o mesmo produto. Testa a estabilidade das estruturas de dados internas.

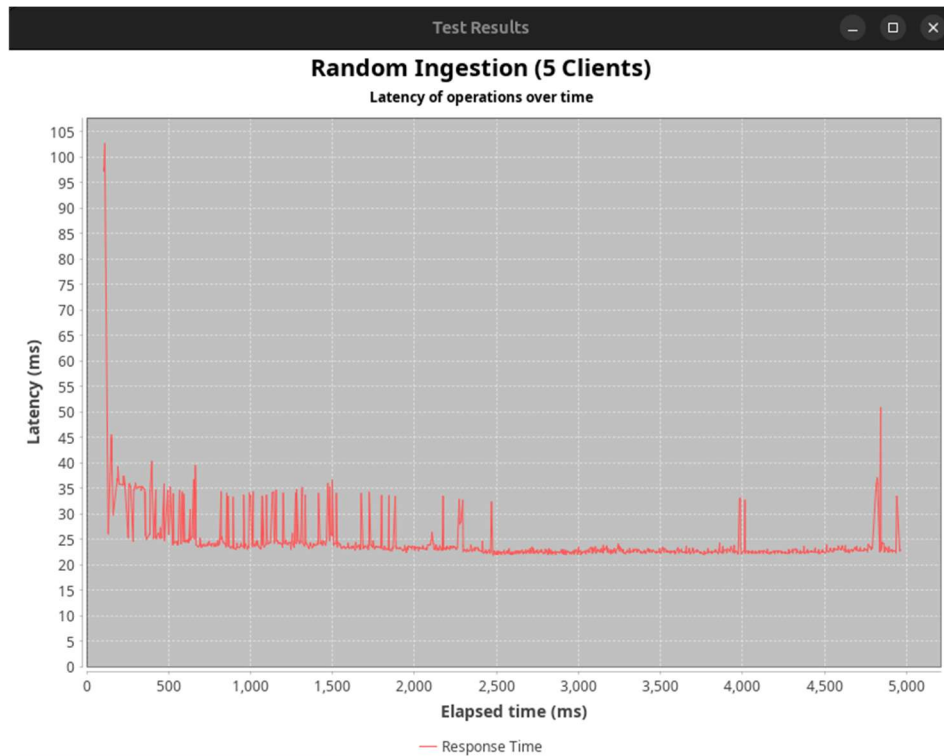
#### 7.1.3 Leitura Intensiva (Agregação)

Múltiplos clientes solicitam agregações simultaneamente. Testa a eficácia do `ReadLock` em permitir paralelismo nas operações de leitura.

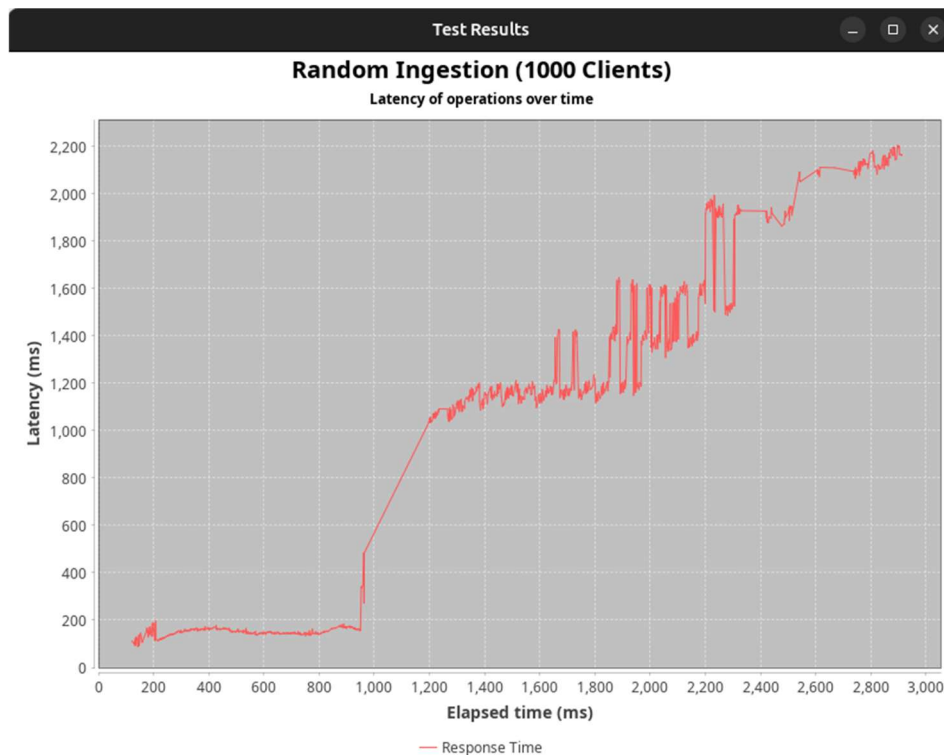
### 7.2 Análise dos Resultados

A análise dos gráficos obtidos permite tirar conclusões importantes sobre o comportamento do sistema.

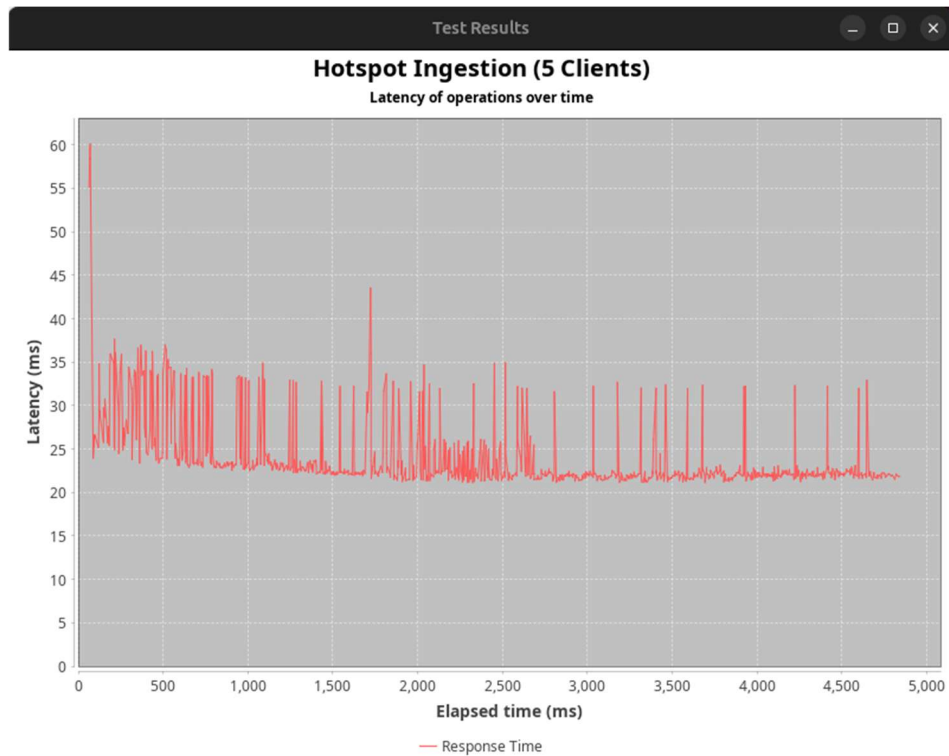
## 7.2.1 Impacto da Concorrência na Escrita (Cenários 1 e 2)



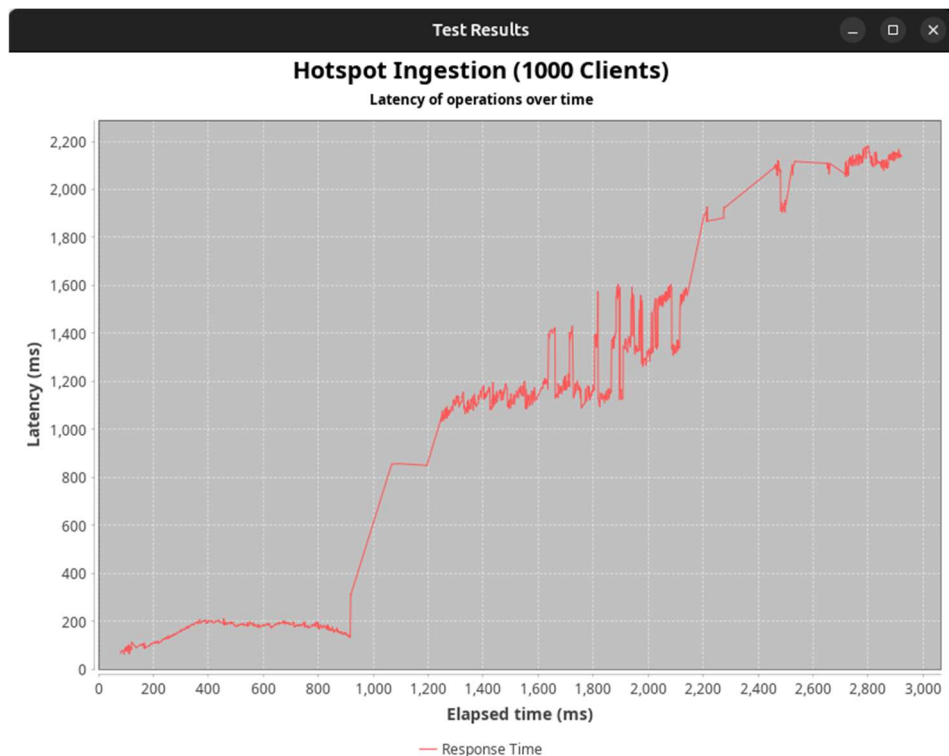
**Figura 1: Random Ingestion (5 Clients)**



**Figura 2: Random Ingestion (1000 Clients)**



**Figura 3: Hotspot Ingestion (5 Clients)**

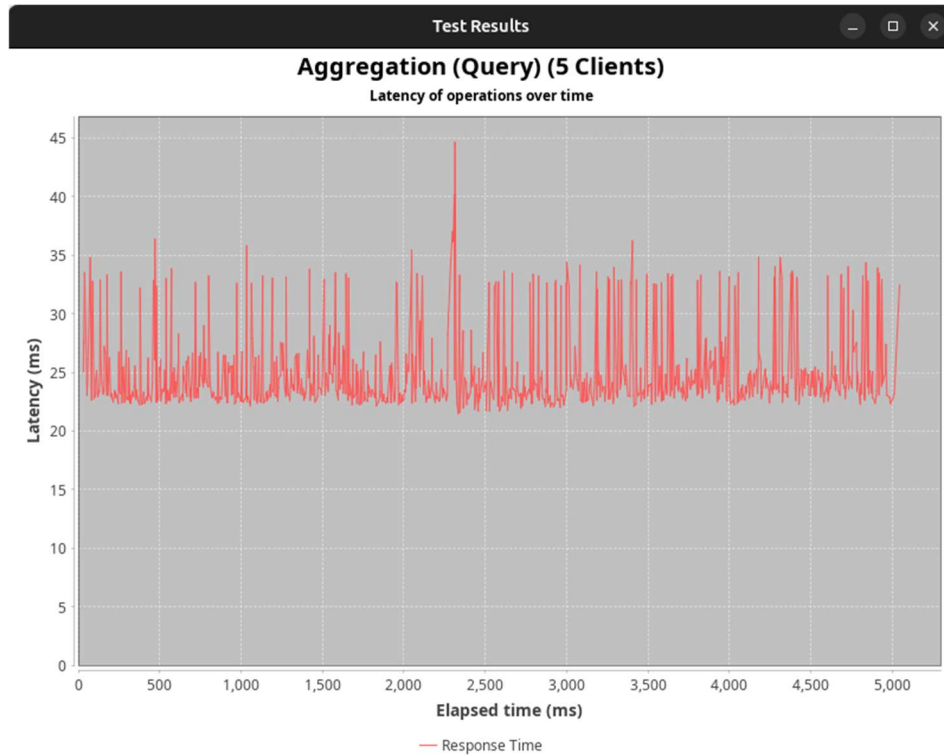


**Figura 4: Hotspot Ingestion (1000 Clients)**

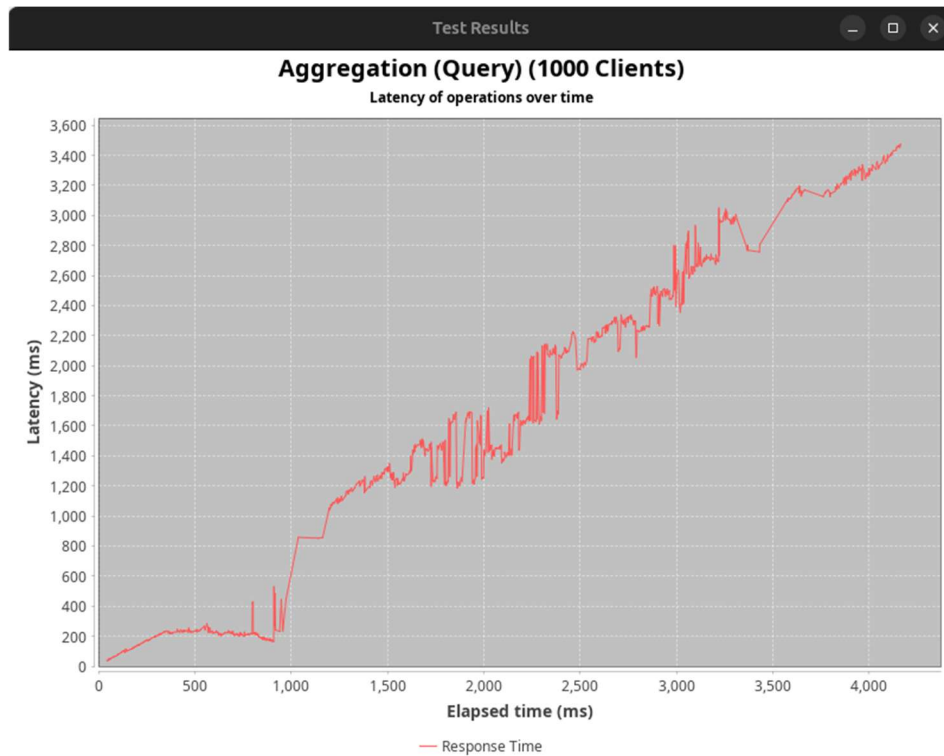
Podemos observar que com uma carga leve de 5 clientes (Figura 1 e 3), a latência manteve-se extremamente baixa (inferior a 30 ms), demonstrando que o overhead do sistema é residual em condições normais. No entanto, com 1000 clientes simultâneos (Figura 2 e 4), a latência aumentou

significativamente, atingindo picos de aproximadamente 2200 ms. Este comportamento é esperado e justifica-se pela decisão de desenho: ao utilizarmos um `WriteLock` exclusivo para o dia corrente, forçamos a serialização das escritas para garantir consistência estrita. As 1000 threads “fazem fila” para adquirir o lock, o que introduz latência, mas previne corrupção de dados.

### 7.2.2 Comportamento na Leitura (Cenário 3)



**Figura 5:** Aggregation (Query) (5 Clients)



**Figura 6:** Aggregation (Query) (1000 Clients)

No teste de agregação com 5 clientes (Figura 5) podemos observar que a latência oscila entre os 20 e os 35 ms. Já no teste com 1000 clientes (Figura 6), observou-se uma latência máxima de aproximadamente 3500 ms superior até à registada na escrita. Embora o uso de `ReadLock` permita que múltiplas threads acedam aos dados simultaneamente sem bloqueio mútuo, a latência aumentou devido ao custo computacional da operação. Enquanto a escrita é uma operação  $O(1)$ , a agregação obriga a iterar sobre históricos de eventos, consumindo mais CPU. Além disso, como o teste cria uma nova conexão TCP para cada pedido, o overhead de estabelecer 1000 conexões e gerir 1000 threads ativas simultaneamente saturou os recursos da máquina de teste (CPU e Sistema Operativo), tornando-se o fator limitante.

## 8. Conclusão

Concluindo, o sistema desenvolvido cumpre integralmente os requisitos propostos. A arquitetura baseada em multiplexagem permitiu suportar clientes multi-threaded de forma robusta e transparente. A estratégia de gestão de memória híbrida (RAM/Disco) assegura escalabilidade para volumes de dados massivos, equilibrando performance através de caching e consistência através de persistência robusta.